CS683: Advanced Computer Architecture — Programming Assignment II

1 Task 1: Stride Pattern Prefetching (SPP)

1.1 Task 1A: Baseline IP Stride

The baseline IP stride prefetcher uses a 24 entry tracker table to record memory access on the basis of IP. On a demand access, the last 8 bits of a 64 bit IP is used to map it to a entry int the tracker table. If a entry matches then the current stride is $\Delta = cl - last_cl$. If $\Delta == last_stride$, then the prefetcher issues D lookahead requests within the 4KB page. If no entry matches, then the LRU tracker is evicted and reallocated.

1.1.1 Limitations of Baseline IP Stride

- Folding 64 bit IPs into a 8 bit hash index with only 24 physical entries will result in a lot of thrashing, (for eg. Trace 1 consist of 121,158 and Trace 2 consists of 94,835 unique IPs).
- Inability of the prefetcher to learn non uniform repeating sequences.
- The prefetcher blindly issues degree 3 prefetches always, this could cause cache pollution and saturate MSHRs during low accuracy phase.

1.2 Task 1B: Stride Pattern Prefetcher (SPP) Microarchitectural Design

To improve upon the baseline IP stride prefetcher we employ the following:

1. **Dual Pattern Classification**: Each IP entry maintains a 2 bit uniform confidence counter (`u_conf`). When `u_conf` < 1, the prefetcher transitions dynamically to **Non Uniform Stride Mode** else it's in **Uniform Stride Mode**.
2. **Signature Tracking & PHT Walk**: Non uniform stride sequences are tracked by a 10 bit history signature S. Observed strides are compressed into 7 bit tokens (s_token), and the signature is updated via $S_{new} = ((S_{old} \ll 2) \oplus s_token)$ & 0x3FF. A 1024 entry PHT maps each signature to its predicted stride and a 2 bit confidence counter (`conf`). Lookahead is achieved done by traversing the learned Markov graph across PHT entries whenever `conf` ≥ 1.
3. **Prefetch Filter**: A circular 32 entry FIFO buffer tracks the recently requested cache lines, which allows us to drop redundant prefetches.

1.2.1 How Did We Decide the Thresholds for each prefetch degrees?

Prefetch accuracy during runtime is evaluated over sliding window of 128 resolved prefetches:

$$\text{Accuracy} = \frac{\Delta\text{pf_useful}}{\Delta\text{pf_useful} + \Delta\text{pf_useless}} \times 100\%$$

The exact percentages were not guess work but rather these were tuned by the use of optuna on a smaller set of instruction count and then we tested them on the default 25M instruction workload to confirm that these values were indeed optimal.

- **Lower Bound** ($< 42\%$): An accuracy below 42% means that more than half of the issued prefetches waste memory bandwidth and displace active cache lines, so prefetching is set to the minimum degree ($D = 1$).
- **Parity Threshold** ($42\% \leq$ **Accuracy** $< 50\%$): In this intermediate range a prefetching of degree 2 is justified.
- **Aggressive Regime** ($50\% \leq$ **Accuracy** $< 65\%$, $65\% \leq$ **Accuracy** $< 80\%$, **and** $\geq 80\%$): Above 50%, the prefetch stream strongly reflects upcoming demand streams, allowing us to prefetch aggressively ($D = 4$, $D = 6$ and $D = 8$ respectively) to hide DRAM latency.

1.3 Hardware Storage Overhead & Bit-Level Breakdown

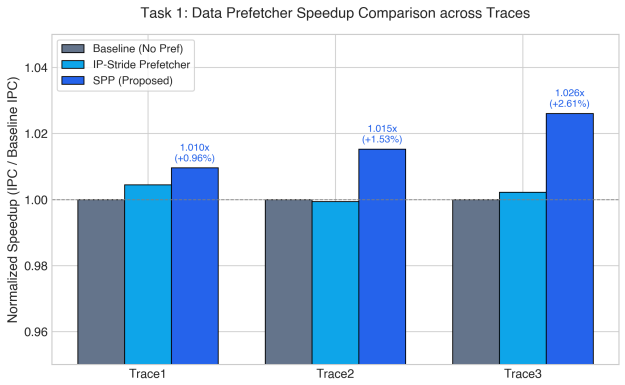
Table 1: Bit Level Breakdown of SPP Prefetcher

Hardware Structure	Field Name	Bits	Role
IP Tracker Table	Tag	16	Upper bits of an IP
(256 Sets × 4 Ways = 1024 Entries)	Valid Bit	1	Indicates Validity
	Last Line Address	58	Cache line address (address without the 6 offset bits)
	Last Stride	8	Strides modeled with 8 bit signed int
	Uniform Confidence	2	2 bit (0-3) conf. counter for uniform strides
	Signature	10	Rolling history of recent strides
	LRU State	2	4 way LRU replacement state
	In Stream Farthest Offset	6	Offset inside page of highest prefetched line (`pf_off`)
	Offset Valid Bit	1	Validity of `pf_off` for current 4KB page (`pf_valid`)
Subtotal per IP Entry		**104 bits**	**13.00 KB for 1024 entries** (106,496 bits)
Pattern History Table (PHT)	Predicted Stride	8	8 bit signed int for pred. delta
(1024 Entries)	Confidence Counter	2	2 bit (0-3) counter (threshold ≥ 1)
Subtotal per PHT Entry		**10 bits**	**1.250 KB for 1024 entries** (10,240 bits)
Prefetch Request Filter	Cache Line Address	64	32 entry buffer of recent line requests
Subtotal for Filter		**64 bits**	**0.250 KB for 32 entries** (2,048 bits)
Accuracy Registers	Useful Snapshot	16	Previous epoch useful
	Useless Snapshot	16	Previous epoch useless
	Active Degree Register	3	Active prefetch degree ($D \in \{1, 2, 4, 6, 8\}$)
Subtotal for Throttler		**35 bits**	**0.004 KB** (35 bits)
Total Hardware Budget Overhead			**14.504 KB (118,819 bits)**

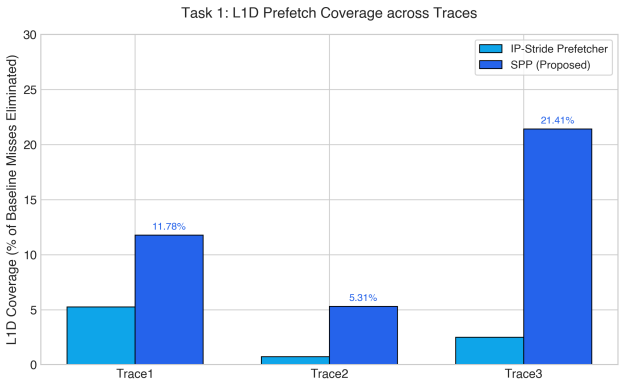
1.4 Task 1 Experimental Results & Analysis

Table 2: Task 1 Performance across Benchmark Traces (25M Warmup + 25M Simulation)

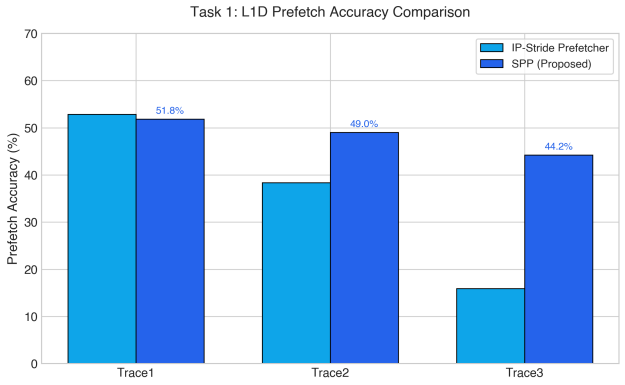
Benchmark	Configuration	IPC	Speedup	L1D MPKI	Accuracy	Coverage	Useful Prefetches
Trace 1	Baseline (No Pref)	0.232264	1.0000x	10.6786	—	—	0
	IP-Stride	0.233313	1.0045x (+0.45%)	10.1180	52.88%	5.25%	16,380
	SPP (Proposed)	**0.234166**	**1.0082x (+0.82%)**	**9.3886**	**51.84%**	**12.08%**	**48,977 (+199%)**
Trace 2	Baseline (No Pref)	0.250806	1.0000x	13.5412	—	—	0
	IP-Stride	0.250650	0.9994x (-0.06%)	13.4406	38.36%	0.74%	4,418
	SPP (Proposed)	**0.254574**	**1.0150x (+1.50%)**	**12.8308**	**49.03%**	**5.25%**	**24,428 (+453%)**
Trace 3	Baseline (No Pref)	0.381942	1.0000x	3.4037	—	—	0
	IP-Stride	0.382799	1.0022x (+0.22%)	3.3189	15.94%	2.49%	2,714
	SPP (Proposed)	**0.394015**	**1.0316x (+3.16%)**	**2.5997**	**44.82%**	**23.62%**	**21,550 (+694%)**



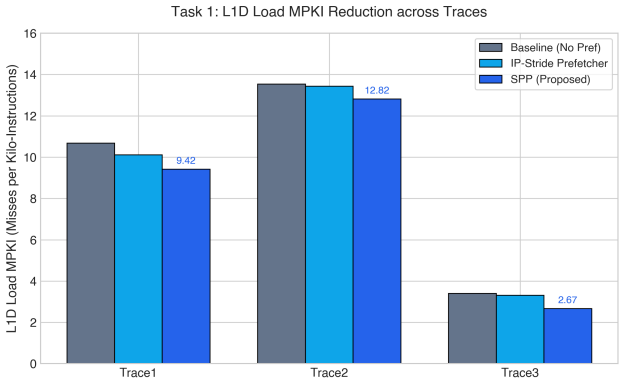
(a) Normalized Speedup (IPC / Baseline IPC)



(b) L1D Coverage (% Misses Eliminated)



(c) L1D Prefetch Accuracy (%)



(d) L1D Load MPKI Reduction

Figure 1: Task 1 SPP Performance Metrics

---

2 Task 2: Instruction-Conscious L2 Replacement

2.1 Task 2A: Instruction Conscious RRIP (I-RRIP)

2.1.1 Code change

- `inc/cache.h`: the declaration becomes `void l2c_update_replacement_state(cpu, set, way, full_addr, ip, victim_addr, type, hit, uint8_t isInstruction)`.
- `src/cache.cc`: the call sites pass the packet's `instruction` flag, so every hit and every fill the policy knows whether the line belongs to the instruction stream.
- `replacement/irrip.l2c_repl`: the flag is stored per line in `l2_is_instruction[set][way]`. `find_victim` needs it as it receives only the set.

Per line the cost is 2 bit RRPV plus one more bit for the instruction flag.

2.1.2 Policy

Table 3: I-RRIP vs RRIP.

Stage	Regular RRIP	I-RRIP
Insertion (miss fill)	all lines at RRPV = 2	instruction line to **1**; data line to 2
Promotion (hit)	all lines to RRPV = 0	instruction line to **0**; data line to **1**
Eviction	first line at RRPV = 3; else age all lines	2 tier (data first)

Eviction proceeds in this order:

1. If a way is invalid, use it.
2. A **data** line with RRPV = 3 is the victim.

3. Otherwise a **data** line with RRPV = 2 (`maxRRPV - 1`) is the victim. This check happens *before* any instruction line is considered.

4. Otherwise any line with RRPV = 3 is the victim. Step 2 has already ruled out data lines at 3, so this can only be an instruction line.

5. Otherwise increase state of every line (RRPV++) and the search repeats.

2.1.3 Reasoning

Insertion Inst. lines enter at RRPV = 1, data at 2. On a single hit it goes to state to RRPV = 0, we insert data lines at RRPV = 2. This gives inst. lines more priority.
Promotion A hit sends on a inst. line goes to RRPV = 0 and a data line only to 1. Again, this is done to prioritize inst. lines over data.
Eviction Our order makes data lines at RRPV ≥ 2 absorb evictions first. Inst. lines are evicted only when no data line is present.

2.1.4 Results

Table 4: I-RRIP vs baseline LRU (25M warm-up + 25M simulation).

Trace	Configuration	IPC	Speedup	L2C Load MPKI	L2 Instr. Load MPKI	L2 Data Load MPKI
Trace 1	Baseline (LRU)	0.2321	1.0000x	17.725	10.367	7.359
Trace 1	I-RRIP	0.2316	0.9979x (-0.21%)	18.793	9.204	9.589
Trace 2	Baseline (LRU)	0.2500	1.0000x	28.550	19.844	8.706
Trace 2	I-RRIP	0.2589	**1.0357x (+3.57%)**	21.067	**9.993**	11.074
Trace 3	Baseline (LRU)	0.3819	1.0000x	3.961	1.127	2.835
Trace 3	I-RRIP	0.3825	1.0016x (+0.16%)	3.940	0.788	3.152

$$\text{Speedup} = \frac{\text{IPC}_{\text{I-RRIP}}}{\text{IPC}_{\text{LRU}}} \tag{1}$$

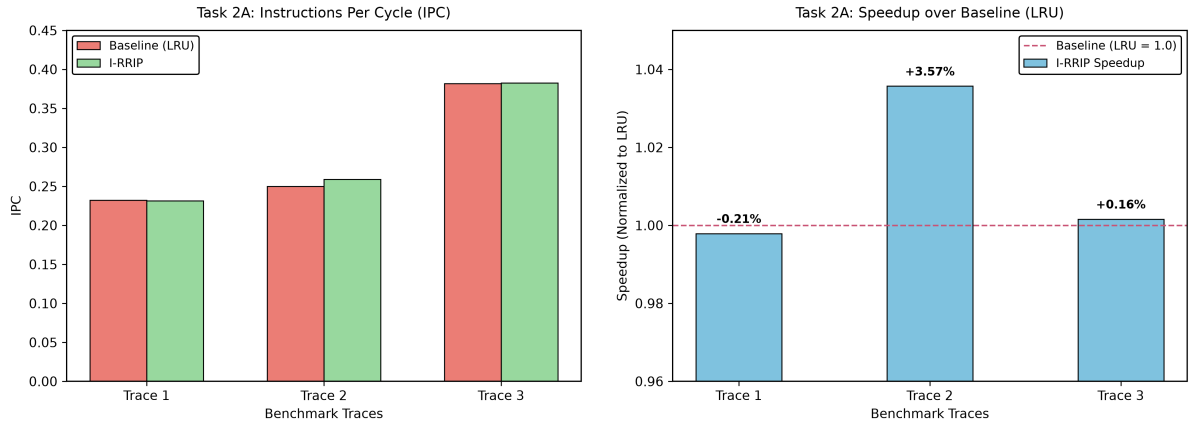


Figure 2: Task 2A: IPC and Speedup over LRU.

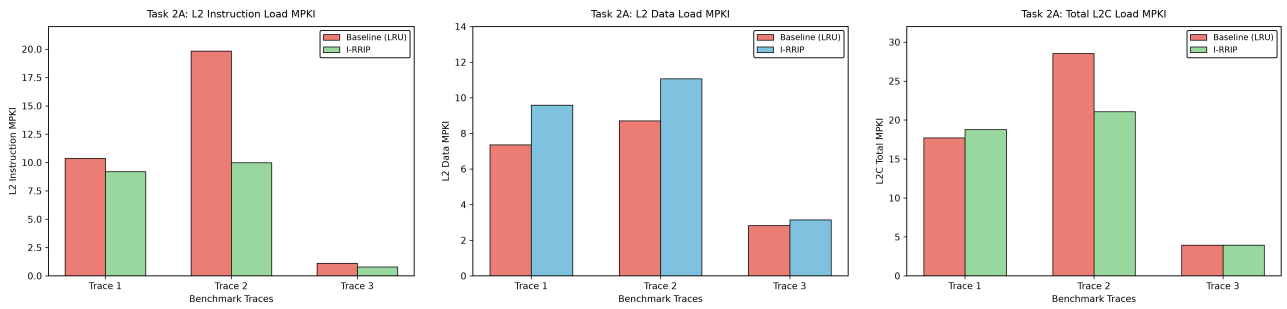


Figure 3: Task 2A: MPKI breakdown (Instruction, Data, Total).

2.2 Task 2B: Dynamic I-RRIP

2.2.1 Mechanism

Implemented in `replacement/dyn_irrip.l2c_repl`.

- **Leader sets.** Set where index is multiple of 16 are leaders. This gives us 64 of the 1024 L2 sets. Leaders always run reg. RRIP regardless of what the rest do.
- **Monitoring.** When a leader set observes a demand event, we update four counters: `instr_acc`, `instr_miss`, `data_acc`, `data_miss`. On every hit and every fill a access is counted on each and a miss is counted on each fill.
- **Decision metric**:

$$\text{score} = \frac{\text{instr_acc} + \text{instr_miss}}{\text{instr_acc} + \text{instr_miss} + \text{data_acc} + \text{data_miss}} \tag{2}$$

  After a leader update, `use_irrip = (score > THRESHOLD)`.
  We use `THRESHOLD = 0.25`.
- **Follower sets** behave as I-RRIP while `use_irrip` is set and as regular RRIP. The flag default is not set, so the cache starts in RRIP mode.
- **Reasoning behind leader's implementation** If leaders ran I-RRIP then inst. protection would change the miss counts. Leaders on plain RRIP give a idea of the workload's instruction load that the controller's own decision does not change.

2.2.2 Sensitivity study of the decision threshold (5 values)

The thresholds 0.15, 0.20, 0.25, 0.30 and 0.35 were tested. The sweep was run with 32 leader sets (stride 32).

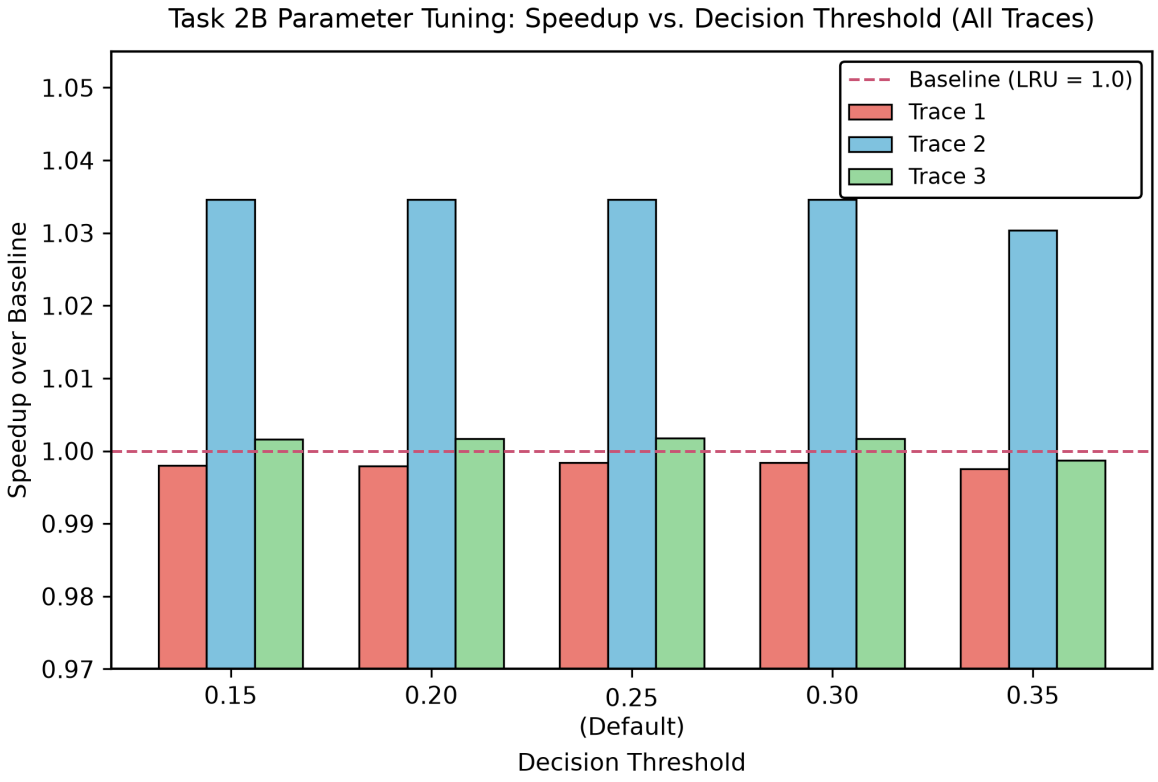


Figure 4: Task 2B: Speedup vs threshold for all traces.

Analysis

- **0.15–0.30.** IPC, speedup and all three MPKIs are almost similar in this threshold range. The decision is thus not very sensitive to the exact threshold in the corresponding range.
- **0.35.** At 0.35 the followers start switching back to plain RRIP and this is evident on all the three traces. (instruction MPKI rises, data MPKI falls, and speedup drops).
- **Overall** The score of these traces lies between about 0.30 and 0.35. Thresholds up to 0.30 prioritize instruction throughout and from 0.35 onwards the same is not followed.

2.2.3 Sensitivity study of the number of leader sets

Keeping the threshold fixed at 0.25.

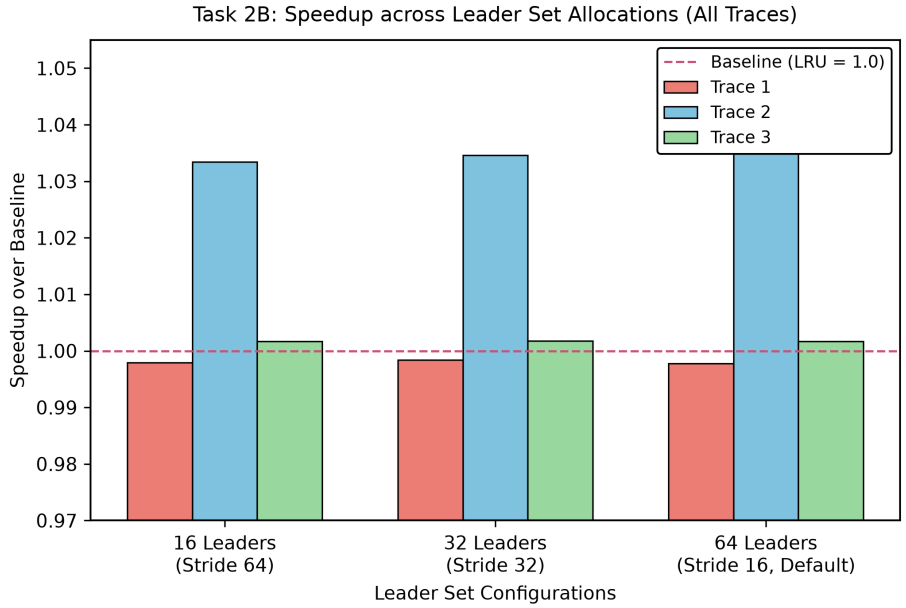


Figure 5: Task 2B: Speedup for 16 / 32 / 64 leader sets.

2.2.4 Reason for the number of leader sets and threshold

Number of leader sets: 64 (stride 16).

- On Trace 2, speedup increases with number of leaders (+3.34%, +3.46%, +3.52% for 16, 32, 64) and along with falling MPKI (10.538, 10.255, 10.115).
- More leaders gives us a better idea of the mix between inst. and data. Inst. occupy limited subset of sets, so with very few leaders the sample can falsely over represent or under represent live instruction traffic.
- The trade off of using more leaders is that those sets never get the instruction bias. At 6.25% of the sets the cost is little, and the trend on Trace 2 favors the bigger sample.

Threshold

- **Threshold = 0.25.** The sweep shows a similar values in the ranges from 0.15 to 0.30 and a drop at 0.35. We chose 0.25 as it sits in the middle of the range and is away from 0.35.

2.2.5 Result: Dynamic I-RRIP (64 leader sets, threshold 0.25)

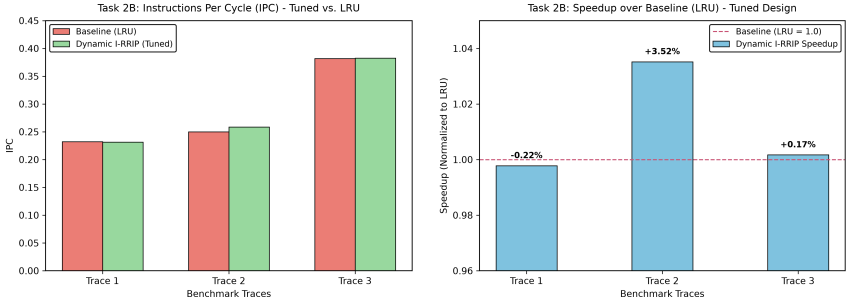


Figure 6: Task 2B: IPC and Speedup over LRU.

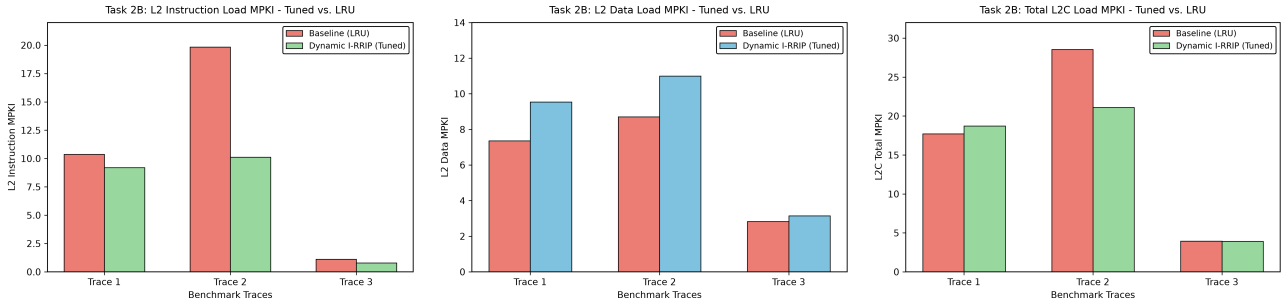


Figure 7: Task 2B: MPKI breakdown (Instruction, Data, Total).

3 Task 3: SPP with Dynamic I-RRIP

This task combines SPP at the L1D cache with the Dynamic I-RRIP policy at the L2 cache to get an improvement over both task 1 and task 2. Our goal is the fetch data aggressively without polluting the L2 cache while making sure we do not evict critical inst. lines.

3.1 Performance Metrics

$$\text{Speedup} = \frac{\text{IPC}_{\text{Task 3}}}{\text{IPC}_{\text{Baseline (LRU)}}} \tag{3}$$

$$\text{L1D Coverage} = \frac{\text{L1D Load Misses}_{\text{Baseline}} - \text{L1D Load Misses}_{\text{Task 3}}}{\text{L1D Load Misses}_{\text{Baseline}}} \tag{4}$$

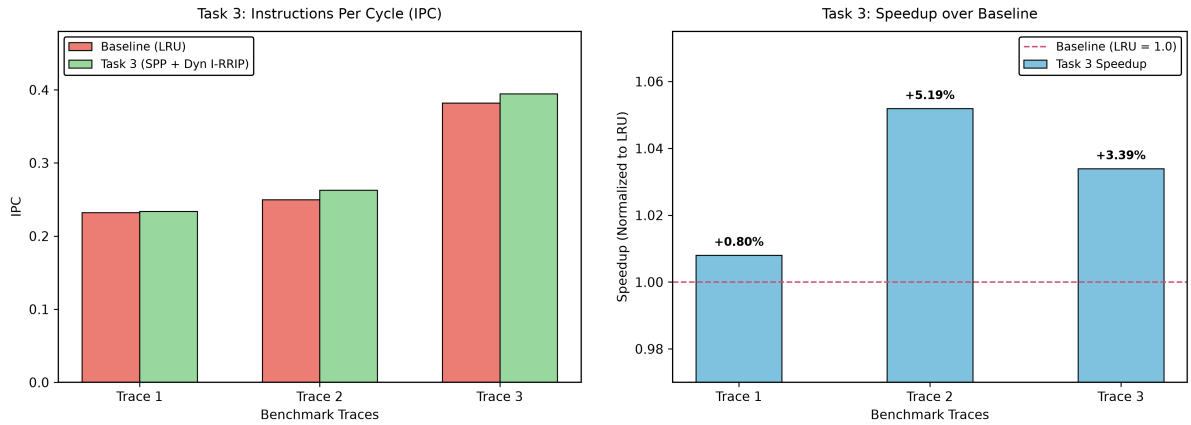


Figure 8: Task 3: IPC and Speedup over Baseline.

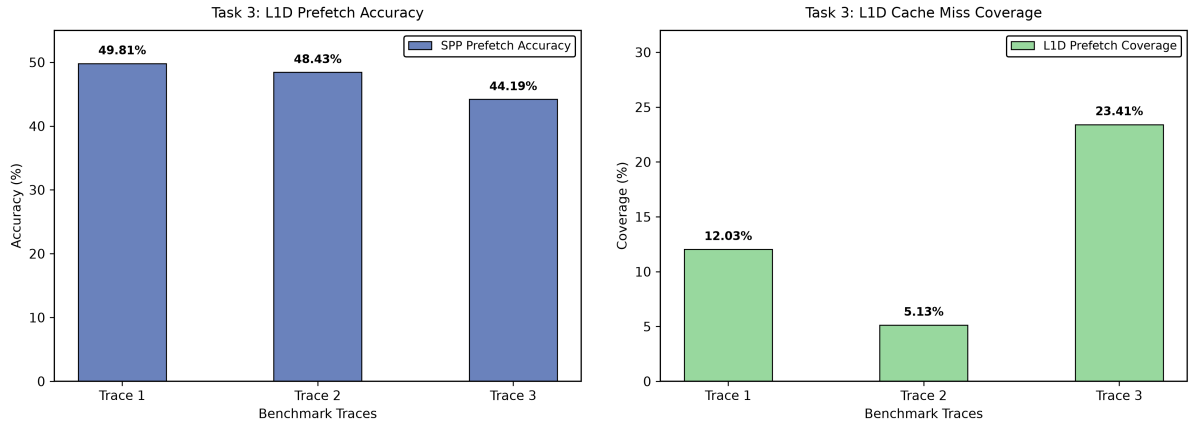


Figure 9: Task 3: SPP L1D Prefetch Accuracy and Cache Miss Coverage.

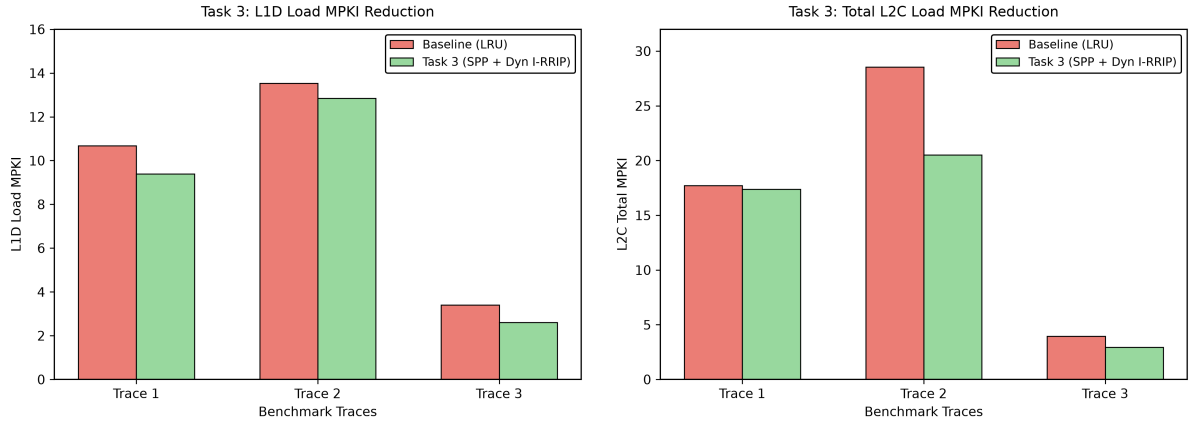


Figure 10: Task 3: L1D Load MPKI and Total L2C Load MPKI Reduction.

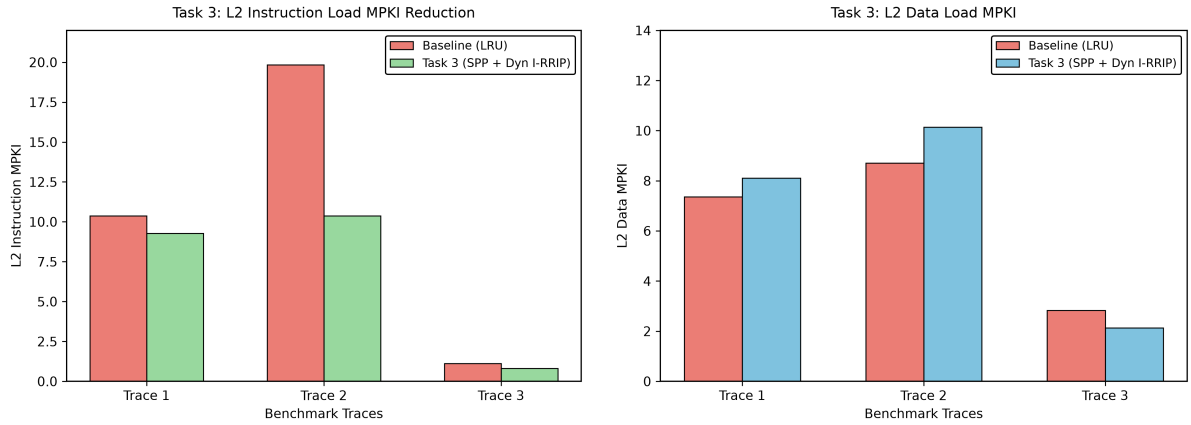


Figure 11: Task 3: L2 Instruction and Data Load MPKI.

3.2 Handling Prefetch Packets in the Replacement Policy

The logic behind the changes are:

- **Excluding the prefetches from the leader counters:** We use set dueling to measure demand type. Because SPP produces a stream of data prefetches (`isInstruction == 0`) and considering them will inflate the data stats falsely. This will reduce the inst. ratio and trick cache into turning off inst. prioritization when it's flooded with traffic that is speculative.
- **Deprioritizing prefetches on insertion:** As our SPP's accuracy is around 44–50%, so half of the lines are never used. If we insert them at the lowest state (RRPV = 3), they becomes the next victim for eviction. This protects live instructions as well as data lines in the cache.
- **Promotion on cache hits:** Speculating traffic shouldn't be able to earn high priority on its own. If during prefetch a line gets a hit which is already in the L2, we promote it to RRPV = 2 only. A line must experience a demand hit to reach protected tiers (RRPV ≤ 1).
- **2 tier eviction:** Data misses can be hidden partially by the processor's O3 execution. Because inaccurate prefetches are inserted at RRPV = 3 and marked as data, our 2 tier eviction policy clears them out first. This protects the instruction lines and gives us **+5.19% speedup** on Trace 2 and **+3.39% speedup** on Trace 3.

---

4 Theoretical Performance Limits

To quantify ceiling for memory optimizations on the given traces we conducted the below experiments:

Table 5: Performance Ceilings across Traces

Configuration	**Trace 1 IPC (Speedup)**	**Trace 2 IPC (Speedup)**	**Trace 3 IPC (Speedup)**
Baseline	0.2323 (1.000×)	0.2508 (1.000×)	0.3819 (1.000×)
PERFECT_L1D (100% L1D Hits)	0.3329 (1.433×)	0.3514 (1.401×)	0.4393 (1.150×)
PERFECT_L2C_INSTR (0 L2 Instr Misses)	0.3313 (1.426×)	0.3207 (1.278×)	0.3983 (1.043×)
PERFECT_L2C_ALL (100% L2 Hits)	0.4177 (1.798×)	0.4196 (1.673×)	0.4468 (1.170×)
PERFECT_L1D + L2C (100% Hits)	**0.4294 (1.849×)**	**0.4303 (1.716×)**	**0.4500 (1.178×)**